

LAPORAN PRAKTIKUM MACHINE LEARNING
ANALISIS IMPLEMENTASI KODE PADA REPOSITORI GITHUB

Repo: ALPHARIET/machine-learning



Disusun oleh:

Nama : Alfarian Adisaputra

NIM : G1A024039

Mata Kuliah : Machine Learning

UNIVERSITAS BENGKULU

INFORMATIKA

2026

BAB I PENDAHULUAN

1.1 Latar Belakang

Machine learning merupakan pendekatan komputasional yang memungkinkan sistem membangun pola dari data untuk melakukan klasifikasi, prediksi, atau pengelompokan. Pemahaman machine learning tidak cukup hanya dari definisi algoritma; implementasi kode memperlihatkan bagaimana data disiapkan, model dibentuk, parameter ditetapkan, model dilatih, dan hasil dievaluasi.

Repositori ALPHARIET/machine-learning berisi rangkaian contoh Python yang mencakup dataset dan pembagian data, beberapa metode supervised learning, regresi, teknik scaling, reduksi dimensi, serta clustering. Halaman repositori menunjukkan 25 berkas Python, yaitu berkas ch2_1 sampai ch2_11, ch3_1 sampai ch3_13, dan splitter.py.

1.2 Tujuan

- Mengidentifikasi fungsi dan alur kerja setiap berkas Python pada repositori.
- Menjelaskan konsep algoritma yang digunakan berdasarkan implementasi aktual.
- Mencatat parameter, dataset, proses transformasi, metrik evaluasi, dan keluaran setiap program.
- Menyusun dokumentasi yang dapat digunakan untuk mencocokkan teori dengan kode dan hasil eksekusi.

1.3 Ruang Lingkup

Laporan membahas seluruh berkas Python yang tersedia pada branch main saat dianalisis. Pembahasan tidak mengubah kode sumber. Hasil numerik pada laporan merupakan hasil eksekusi ulang dengan parameter yang sama pada lingkungan Python yang tersedia pada saat penyusunan, sedangkan analisis fungsi setiap baris disusun dari kode sumber repositori.

BAB II HASIL DAN PEMBAHASAN

Repository dibangun dengan pola implementasi yang konsisten. Banyak file menggunakan dataset Iris untuk supervised learning, dataset sintetis make_blobs untuk contoh preprocessing dan clustering, dataset digits untuk reduksi dimensi, serta breast cancer dan two moons untuk demonstrasi pengaruh scaling dan clustering berbasis kepadatan. Kode sumber juga memperlihatkan penggunaan scikit-learn seperti KNeighborsClassifier, LogisticRegression, GaussianNB, DecisionTreeClassifier, RandomForestClassifier, SVC, MLPClassifier, LinearRegression, Ridge, PCA, NMF, TSNE, KMeans, AgglomerativeClustering, dan DBSCAN.

Ringkasan Hasil Eksekusi

Berkas	Fungsi	Metode / parameter utama	Hasil utama
ch2_1_dataset.py	Persiapan dataset	Iris; train/test 112/38	Ukuran X_train 112×4; X_test 38×4
ch2_2_knn.py	Klasifikasi	KNN, k=3	Akurasi 1.0000
ch2_3_logistic_regression.py	Klasifikasi	Logistic Regression	Akurasi 1.0000
ch2_4_naive_bayes.py	Klasifikasi	Gaussian NB	Akurasi 0.9737
ch2_5_decision_tree.py	Klasifikasi	Tree, max_depth=3	Akurasi 0.9474
ch2_6_random_forest.py	Klasifikasi	100 trees	Akurasi 0.9474
ch2_7_svm.py	Klasifikasi	RBF + StandardScaler	Akurasi 0.9737
ch2_8_mlp.py	Klasifikasi	1 hidden	Akurasi

Berkas	Fungsi	Metode / parameter utama	Hasil utama
		layer, 10 neuron	0.9737
ch2_9_perbandingan_model.py	Perbandingan	6 model	0.9474–1.0000
ch2_10_linear_regression.py	Regresi	Linear Regression	RMSE 1.0457
ch2_11_ridge_regression.py	Regresi	Ridge, alpha=1	RMSE 1.0454
ch3_1_dataset_2d.py	Data sintetis	make_blobs, 3 cluster	200 sampel, 2 fitur
ch3_2_standard_scaler.py	Scaling	StandardScaler	mean $\approx$ 0; std $\approx$ 1
ch3_3_minmax_scaler.py	Scaling	MinMaxScaler	min=0; max=1
ch3_4_robust_scaler.py	Scaling	RobustScaler	5 contoh data ditampilkan
ch3_5_normalizer.py	Normalisasi	Normalizer	norm vektor pertama=1
ch3_6_scaling_knn.py	Eksperimen	KNN sebelum/sesudah scaling	0.9231 $\rightarrow$ 0.9441
ch3_7_pca.py	Reduksi dimensi	PCA 64 $\rightarrow$ 2	EVR total $\approx$ 0.21595
ch3_8_nmf.py	Reduksi dimensi	NMF, 10 komponen	1797 $\times$ 64 $\rightarrow$ 1797 $\times$ 10
ch3_9_tsne.py	Visualisasi/manifold	t-SNE, 500 data	500 $\times$ 64 $\rightarrow$ 500 $\times$ 2
ch3_10_kmeans.py	Clustering	K-Means, k=3	Silhouette $\approx$ 0.4440

Berkas	Fungsi	Metode / parameter utama	Hasil utama
ch3_11_agglomerative.py	Clustering	Agglomerative, k=3	Silhouette≈0.4136
ch3_12_dbscan.py	Clustering	DBSCAN, eps=0.3	2 cluster, tanpa noise
ch3_13_perbandingan_clustering.py	Perbandingan	3 algoritma	ARI KMeans≈0.5016; Agglo≈0.6065; DBSCAN=1.0000
splitter.py	Kode pendukung	Generator file latihan	Membentuk berkas dari template

2.1 ch2_1_dataset.py – Persiapan Dataset Iris

Tujuan program ini adalah menyiapkan dataset Iris sebagai dasar untuk contoh supervised learning berikutnya. Program mengambil fitur dari iris.data dan label kelas dari iris.target, kemudian membagi data menggunakan train_test_split dengan random_state=0 dan stratify=y. Dengan stratifikasi, proporsi kelas target dipertahankan pada data latih dan data uji. Kode mencetak ukuran masing-masing bagian, tanpa melatih model.

Analisis implementasi

- Dataset berisi 150 sampel dengan 4 fitur; kode menggunakan seluruh fitur sebagai X dan kelas sebagai y.
- Pembagian menghasilkan 112 data latih dan 38 data uji pada eksekusi yang digunakan untuk laporan.
- random_state=0 membuat pembagian dapat direproduksi, sedangkan stratify=y menjaga representasi kelas.

Hasil dan kesimpulan

Output	Nilai
Ukuran X_train	(112, 4)
Ukuran X_test	(38, 4)

Kesimpulannya, file ini berfungsi sebagai tahap persiapan data yang dipakai berulang oleh enam contoh klasifikasi dan satu contoh perbandingan model.

Sumber kode: github.com/ALPHARIET/machine-learning/ch2_1_dataset.py

2.2 ch2_2_knn.py – K-Nearest Neighbors (KNN)

KNN melakukan klasifikasi berdasarkan kedekatan suatu data terhadap tetangga terdekat. Kode menetapkan `n_neighbors=3`, melatih model pada `X_train` dan `y_train`, lalu menghitung akurasi pada `X_test` serta menampilkan 10 prediksi pertama. Implementasi tidak melakukan scaling karena keempat fitur Iris berada pada skala yang relatif sejenis.

Hasil

Parameter / Output	Nilai
Parameter k	3
Akurasi test	1.0000

Kesimpulan

Kinerja pada pembagian data ini mencapai 100%, sehingga seluruh data uji terklasifikasi dengan benar pada eksekusi laporan.

Sumber kode: github.com/ALPHARIET/machine-learning/ch2_2_knn.py

2.3 ch2_3_logistic_regression.py – Logistic Regression

Program menggunakan `LogisticRegression(max_iter=1000)` untuk klasifikasi multiclass pada Iris. Parameter `max_iter` diperbesar agar proses optimasi memiliki batas iterasi yang cukup. Model dilatih dengan `X_train` dan `y_train` dan skor dihitung pada data uji.

Hasil

Parameter / Output	Nilai
Parameter	<code>max_iter=1000</code>

Parameter / Output	Nilai
Akurasi test	1.0000

Kesimpulan

Pada split yang sama, model menghasilkan akurasi 100%.

Sumber kode: github.com/ALPHARIET/machine-learning/ch2_3_logistic_regression.py

2.4 ch2_4_naive_bayes.py – Gaussian Naive Bayes

Program menggunakan GaussianNB. Model mempelajari distribusi Gaussian tiap fitur untuk kelas dan mengasumsikan independensi bersyarat antarfitur. Kode langsung fit pada data Iris tanpa preprocessing tambahan.

Hasil

Parameter / Output	Nilai
Model	GaussianNB
Akurasi test	0.9737

Kesimpulan

Hasil menunjukkan 37 dari 38 data uji terklasifikasi benar pada eksekusi laporan.

Sumber kode: github.com/ALPHARIET/machine-learning/ch2_4_naive_bayes.py

2.5 ch2_5_decision_tree.py – Decision Tree

Decision TreeClassifier dibatasi dengan max_depth=3 dan random_state=0. Setelah pelatihan, kode mencetak akurasi dan menggambar struktur pohon menggunakan plot_tree dengan nama fitur dan nama kelas Iris.

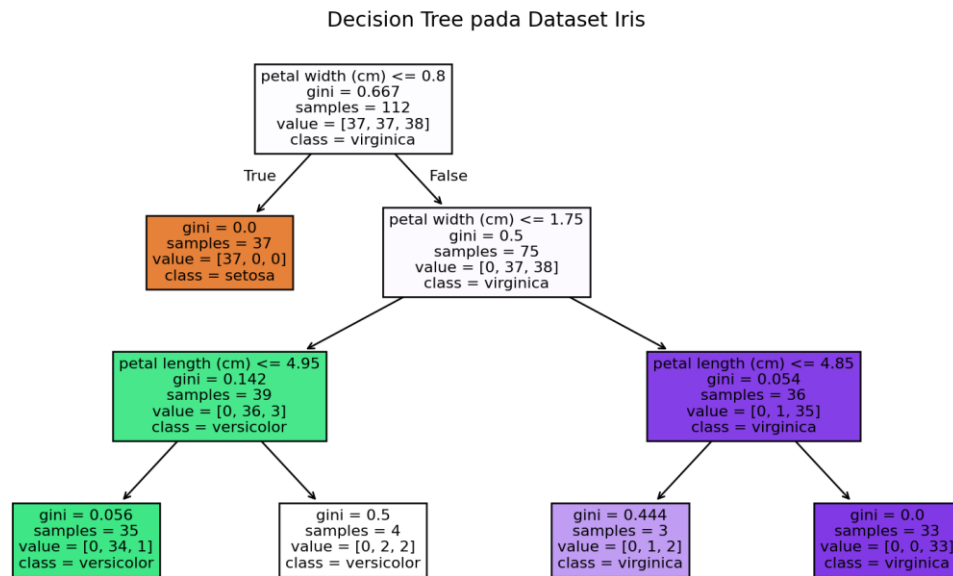
Hasil

Parameter / Output	Nilai
max_depth	3
Akurasi test	0.9474

Kesimpulan

Model menghasilkan akurasi 94,74%. Pembatasan kedalaman menjaga pohon tetap ringkas dan mudah divisualisasikan.

Sumber kode: github.com/ALPHARIET/machine-learning/ch2_5_decision_tree.py



Gambar 1. Visualisasi pohon keputusan sesuai konfigurasi kode.

2.6 ch2_6_random_forest.py – Random Forest

Random Forest membangun 100 pohon keputusan melalui `n_estimators=100` dengan `random_state=0`. Selain skor akurasi, program mencetak `feature_importances_` untuk melihat kontribusi relatif setiap fitur terhadap keputusan ensemble.

Hasil

Parameter / Output	Nilai
n_estimators	100
Akurasi test	0.9474
Petal width	0.4540
Petal length	0.4135
Sepal length	0.1041
Sepal width	0.0285

Kesimpulan

Dari feature importance, petal width dan petal length menjadi dua fitur dengan bobot terbesar pada model ini.

Sumber kode: github.com/ALPHARIET/machine-learning/ch2_6_random_forest.py

2.7 ch2_7_svm.py – Support Vector Machine (SVM)

Program menambahkan StandardScaler sebelum membangun SVC dengan kernel RBF, C=1, dan gamma="scale". Data latih di-fit pada scaler, sedangkan data uji hanya ditransformasi menggunakan parameter scaler dari data latih. Pola ini menghindari kebocoran informasi dari test set ke proses scaling.

Hasil

Parameter / Output	Nilai
Kernel	RBF
C	1
Akurasi test	0.9737

Kesimpulan

Akurasi mencapai 97,37% pada data uji.

Sumber kode: github.com/ALPHARIET/machine-learning/ch2_7_svm.py

2.8 ch2_8_mlp.py – Neural Network / MLP

Program menggunakan MLPClassifier dengan satu hidden layer berisi 10 neuron, max_iter=1000, dan random_state=0. Data juga distandardisasi lebih dulu. Model dilatih pada data hasil scaling dan dievaluasi menggunakan score pada test set.

Hasil

Parameter / Output	Nilai
Hidden layer	(10,)
Iterasi aktual	971
Akurasi test	0.9737

Kesimpulan

Model berhenti pada 971 iterasi dan menghasilkan akurasi 97,37%. Hasil menunjukkan preprocessing dan optimasi iteratif menjadi bagian penting dari implementasi jaringan saraf.

Sumber kode: github.com/ALPHARIET/machine-learning/ch2_8_mlp.py

2.9 ch2_9_perbandingan_model.py – Perbandingan Model Klasifikasi

Program menyiapkan enam model, yaitu KNN, Logistic Regression, Naive Bayes, Decision Tree, Random Forest, dan SVM. Berbeda dari file model tunggal, script ini melakukan loop terhadap dictionary model dan mencetak skor dengan empat angka desimal. Ada satu detail metodologis penting: hanya SVM yang dilatih pada data yang telah di-standardisasi; lima model lainnya menggunakan data mentah. Dengan demikian, hasil menunjukkan perbandingan implementasi pada konfigurasi tersebut, bukan perbandingan yang sepenuhnya seragam dari sisi preprocessing.

Model	Akurasi test
KNN	1.0000
Logistic Regression	1.0000
Naive Bayes	0.9737
Decision Tree	0.9474
Random Forest	0.9474
SVM	0.9737

Pada eksekusi yang digunakan, KNN dan Logistic Regression menghasilkan akurasi tertinggi sebesar 1,0000. Temuan ini hanya menggambarkan satu train-test split, sehingga tidak cukup untuk menyimpulkan keunggulan umum suatu model pada dataset lain atau split yang berbeda.

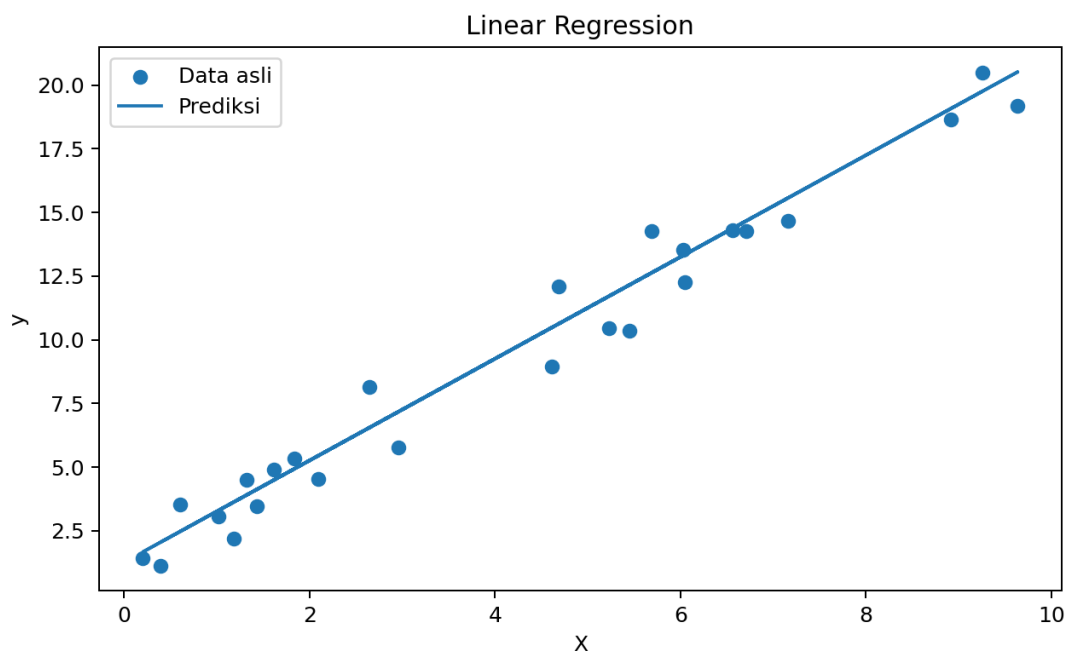
Sumber kode: github.com/ALPHARIET/machine-learning/ch2_9_perbandingan_model.py

2.10 ch2_10_linear_regression.py – Linear Regression

Program membangun data regresi sintetis menggunakan persamaan $y=2x+1$ ditambah noise acak. Data kemudian dibagi menjadi data latih dan uji. LinearRegression dilatih pada data latih, menghasilkan prediksi pada data uji, menghitung koefisien, intercept, serta RMSE. Program juga membuat scatter plot data uji dan garis prediksi.

Output	Nilai
Koefisien	1.99789
Intercept	1.26731
RMSE	1.04567

Koefisien yang diperoleh mendekati koefisien teoritis 2 pada data pembentuk, sedangkan intercept berada di sekitar 1. Nilai RMSE sekitar 1,046 menunjukkan besarnya kesalahan prediksi rata-rata dalam satuan target pada data uji. Karena data mengandung noise, hasil estimasi tidak harus persis sama dengan persamaan pembangkit data.



Gambar 2. Data uji dan garis prediksi Linear Regression.

Sumber kode: github.com/ALPHARIET/machine-learning/ch2_10_linear_regression.py

2.11 ch2_11_ridge_regression.py – Ridge Regression

Ridge Regression menggunakan data sintetis yang sama dengan Linear Regression, tetapi menambahkan regularisasi L2 melalui Ridge(alpha=1.0). Kode hanya mencetak RMSE, sehingga fokus program adalah menunjukkan bagaimana regularisasi dapat digunakan pada model regresi.

Output	Nilai
alpha	1.0
RMSE	1.04544

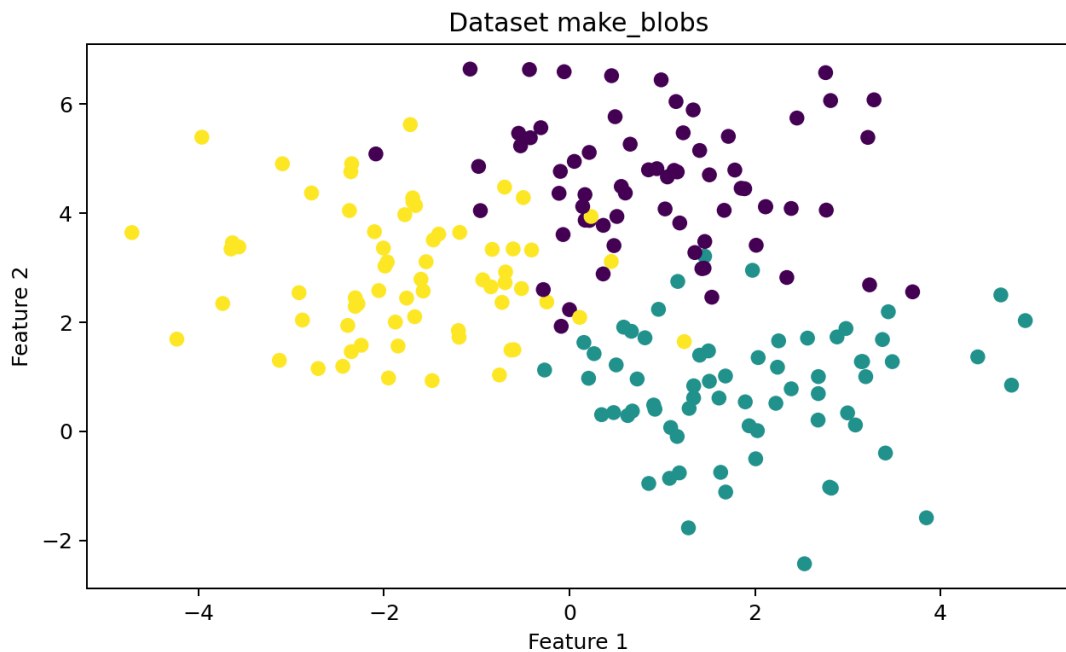
RMSE Ridge pada eksekusi ini sedikit lebih kecil daripada Linear Regression (sekitar 1,04544 dibanding 1,04567). Selisihnya sangat kecil karena hanya ada satu fitur dan dataset relatif sederhana; contoh ini lebih berfungsi untuk memperlihatkan mekanisme regularisasi daripada menghasilkan perbandingan menyeluruh.

Sumber kode: github.com/ALPHARIET/machine-learning/ch2_11_ridge_regression.py

2.12 ch3_1_dataset_2d.py – Dataset 2D dengan make_blobs

Program membuat 200 titik data dua dimensi dengan tiga pusat cluster, random_state=0, dan cluster_std=1.2 menggunakan make_blobs. Label y_blob digunakan untuk memberi warna pada scatter plot. Dataset ini menjadi fondasi untuk contoh StandardScaler, MinMaxScaler, RobustScaler, Normalizer, K-Means, dan Agglomerative Clustering pada berkas berikutnya.

Parameter	Nilai
n_samples	200
centers	3
cluster_std	1.2
Dimensi	2 fitur



Gambar 3. Dataset sintetis make_blobs.

Kesimpulannya, program ini menyiapkan dataset yang memiliki struktur cluster yang mudah divisualisasikan dan sesuai untuk demonstrasi metode clustering.

Sumber kode: github.com/ALPHARIET/machine-learning/ch3_1_dataset_2d.py

2.13 ch3_2_standard_scaler.py – StandardScaler

StandardScaler mentransformasikan setiap fitur dengan mengurangi rata-rata dan membaginya dengan standar deviasi. Program menerapkannya pada `X_blob` lalu memeriksa rata-rata dan standar deviasi hasil transformasi.

Output	Nilai
Rata-rata fitur 1	≈ 0
Rata-rata fitur 2	≈ 0
Std fitur 1	≈ 1
Std fitur 2	≈ 1

Hasil sesuai tujuan StandardScaler: pusat data berada di sekitar nol dan simpangan bakunya satu. Transformasi ini bermanfaat untuk algoritma yang sensitif terhadap skala fitur.

Sumber kode: github.com/ALPHARIET/machine-learning/ch3_2_standard_scaler.py

2.14 ch3_3_minmax_scaler.py – MinMaxScaler

MinMaxScaler memetakan setiap fitur ke rentang tertentu. Dalam konfigurasi default scikit-learn, data dipetakan ke rentang 0 sampai 1. Kode memeriksa nilai minimum dan maksimum setelah transformasi.

Output	Nilai
Minimum tiap fitur	[0.0, 0.0]
Maksimum tiap fitur	[1.0, 1.0]

Hasil menunjukkan pemetaan berhasil menghasilkan batas minimum 0 dan maksimum 1 pada kedua fitur.

Sumber kode: github.com/ALPHARIET/machine-learning/ch3_3_minmax_scaler.py

2.15 ch3_4_robust_scaler.py – RobustScaler

RobustScaler menggunakan statistik berbasis median dan rentang antarkuartil sehingga lebih tahan terhadap pengaruh outlier dibanding scaling berbasis mean dan standar deviasi. Program mencetak lima baris pertama dari data hasil transformasi.

Baris	Feature 1	Feature 2
1	-0.8695	0.4702
2	0.0089	0.6517
3	0.2360	0.7452
4	-0.7320	0.3450
5	0.3007	1.1446

Contoh keluaran memperlihatkan nilai fitur telah berubah pusat dan skalanya. File ini terutama berfungsi memperkenalkan alternatif scaling ketika data berpotensi mengandung pencilan.

Sumber kode: github.com/ALPHARIET/machine-learning/ch3_4_robust_scaler.py

2.16 ch3_5_normalizer.py – Normalizer

Normalizer berbeda dari tiga scaler sebelumnya karena normalisasi dilakukan per-sampel atau per-vektor, bukan dengan menyesuaikan distribusi setiap fitur secara global. Kode menghitung norma Euclidean dari baris pertama setelah transformasi.

Output	Nilai
Norma vektor pertama	1.0

Norma 1 menunjukkan setiap baris telah dinormalisasi menjadi vektor satuan. Teknik seperti ini berguna saat arah vektor lebih penting daripada besar absolutnya.

Sumber kode: github.com/ALPHARIET/machine-learning/ch3_5_normalizer.py

2.17 ch3_6_scaling_knn.py – Pengaruh Scaling pada KNN

Program menggunakan dataset breast cancer, membagi data latih dan uji, kemudian membangun dua KNN dengan `n_neighbors=3`. Model pertama menggunakan fitur asli; model kedua menggunakan fitur yang telah diproses `StandardScaler`. Kode mencetak skor kedua pendekatan untuk memperlihatkan dampak perbedaan skala terhadap metode berbasis jarak.

Konfigurasi	Akurasi test
Tanpa scaling	0.9231
Dengan scaling	0.9441
Perubahan	+0.0210 ($\approx$ 2,10 poin persentase)

Pada data yang diuji, scaling meningkatkan akurasi KNN sekitar 2,10 poin persentase. Hasil ini mendukung alasan bahwa skala fitur perlu diperhatikan pada algoritma yang menggunakan jarak, meskipun besarnya pengaruh bergantung pada dataset.

Sumber kode: github.com/ALPHARIET/machine-learning/ch3_6_scaling_knn.py

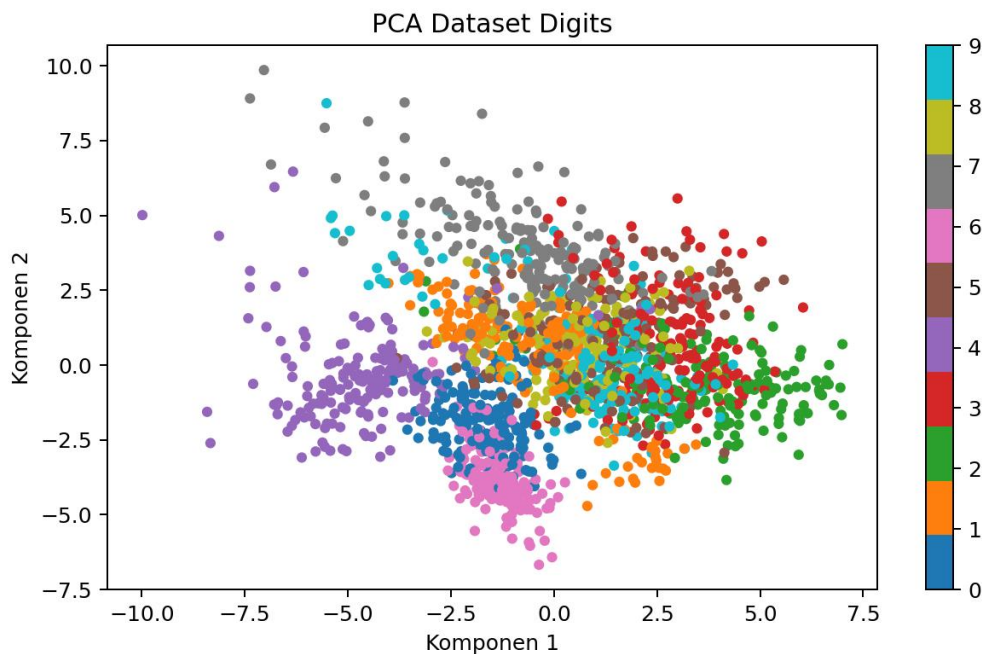
2.18 ch3_7_pca.py – Principal Component Analysis (PCA)

PCA digunakan untuk mereduksi data digits dari 64 fitur menjadi dua komponen. Sebelum PCA, fitur distandardisasi dengan `StandardScaler`. Program

mencetak ukuran sebelum dan sesudah reduksi serta explained variance ratio, kemudian memvisualisasikan dua komponen pertama dan memberi warna berdasarkan label digit.

Output	Nilai
Ukuran sebelum PCA	(1797, 64)
Ukuran sesudah PCA	(1797, 2)
EVR Komponen 1	0.12034
EVR Komponen 2	0.09561
Total EVR dua komponen	0.21595

Dua komponen pertama menyimpan sekitar 21,60% variasi data. Dengan demikian, reduksi ke dua dimensi sangat membantu visualisasi tetapi hanya mempertahankan sebagian variasi keseluruhan 64 dimensi.



Gambar 4. Proyeksi dataset digits ke dua komponen PCA.

Sumber kode: github.com/ALPHARIET/machine-learning/ch3_7_pca.py

2.19 ch3_8_nmf.py – Non-negative Matrix Factorization (NMF)

Program memuat dataset digits, membuat versi terstandarisasi, tetapi secara khusus mempertahankan digits.data asli untuk NMF karena NMF mensyaratkan data non-negatif. NMF dibentuk dengan `n_components=10`,

init="nndsvda", random_state=0, dan max_iter=500. Hasil transformasi berukuran 1797×10 .

Output	Nilai
Input ke NMF	(1797, 64)
n_components	10
Output	(1797, 10)
Catatan	Data non-negatif asli digunakan

Implementasi menunjukkan perhatian penting terhadap prasyarat algoritma: data yang dipakai NMF bukan `X_digits_scaled`, melainkan `digits.data` yang tidak mengandung nilai negatif. Ini adalah perbedaan konseptual penting dibanding PCA pada file sebelumnya.

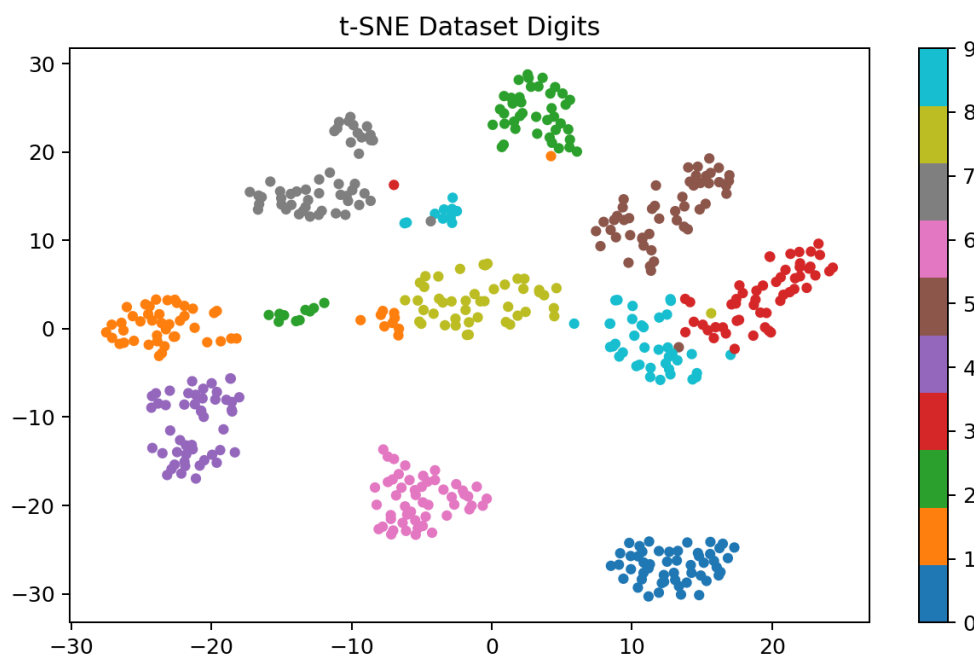
Sumber kode: github.com/ALPHARIET/machine-learning/ch3_8_nmf.py

2.20 ch3_9_tsne.py – t-SNE

t-SNE digunakan untuk menurunkan representasi dataset `digits` ke dua dimensi untuk visualisasi. Kode menggunakan data yang sudah distandardisasi, tetapi hanya mengambil 500 sampel pertama agar waktu komputasi lebih ringan. Parameter utama meliputi `n_components=2`, `random_state=0`, `init="pca"`, dan `learning_rate="auto"`.

Output	Nilai
Jumlah sampel yang diproses	500
Dimensi awal	64
Dimensi akhir	2
init	pca
learning_rate	auto

Hasil utama file ini berupa koordinat dua dimensi untuk keperluan visualisasi. Karena t-SNE bersifat nonlinier dan fokus pada struktur kedekatan lokal, hasil visual dapat berubah ketika seed atau konfigurasi diubah; `random_state=0` digunakan agar contoh dapat direproduksi.



Gambar 5. Visualisasi t-SNE pada 500 sampel pertama dataset digits.

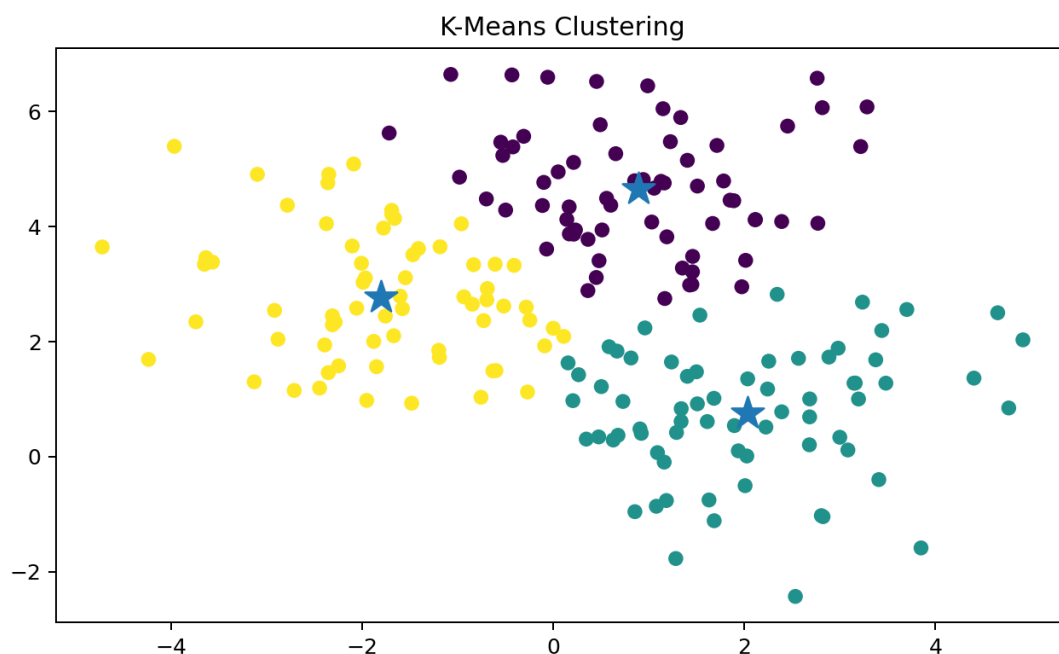
Sumber kode: github.com/ALPHARIET/machine-learning/ch3_9_tsne.py

2.21 ch3_10_kmeans.py – K-Means

K-Means membagi data `make_blobs` menjadi tiga cluster dengan `KMeans(n_clusters=3, random_state=0, n_init=10)`. Hasil label cluster dihitung menggunakan `fit_predict`, kemudian kualitas pemisahan cluster dinilai menggunakan `silhouette_score`. Titik pusat cluster juga divisualisasikan sebagai marker bintang.

Output	Nilai
Label cluster	[0, 1, 2]
Silhouette score	0.4440
n_clusters	3
n_init	10

Silhouette sekitar 0,444 menunjukkan pemisahan cluster berada pada tingkat sedang untuk konfigurasi data tersebut. Kode memperlihatkan alur inti K-Means: inisialisasi jumlah cluster, fitting, pengambilan label, evaluasi, lalu visualisasi centroid.



Gambar 6. Hasil K-Means dan posisi centroid.

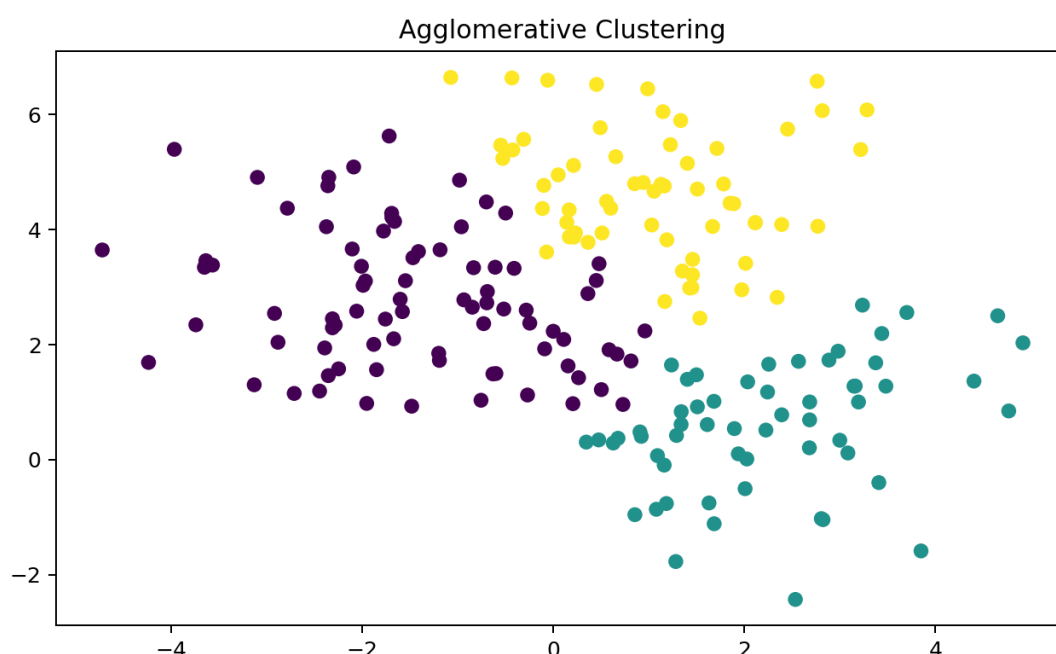
Sumber kode: github.com/ALPHARIET/machine-learning/ch3_10_kmeans.py

2.22 ch3_11_agglomerative.py – Agglomerative Clustering

AgglomerativeClustering digunakan dengan `n_clusters=3` pada dataset `make_blobs`. Program langsung mendapatkan label cluster melalui `fit_predict` dan menghitung silhouette score, lalu memvisualisasikan hasilnya. Berbeda dari K-Means, agglomerative clustering membentuk struktur hierarkis yang digabung bertahap hingga jumlah cluster yang ditentukan tercapai.

Output	Nilai
<code>n_clusters</code>	3
Silhouette score	0.4136
Label cluster	[0, 1, 2]

Silhouette 0,4136 sedikit lebih rendah daripada K-Means pada dataset yang sama. Perbandingan ini menggambarkan bahwa algoritma yang berbeda dapat membentuk pembagian cluster yang berbeda walaupun jumlah cluster ditetapkan sama.



Gambar 7. Hasil Agglomerative Clustering.

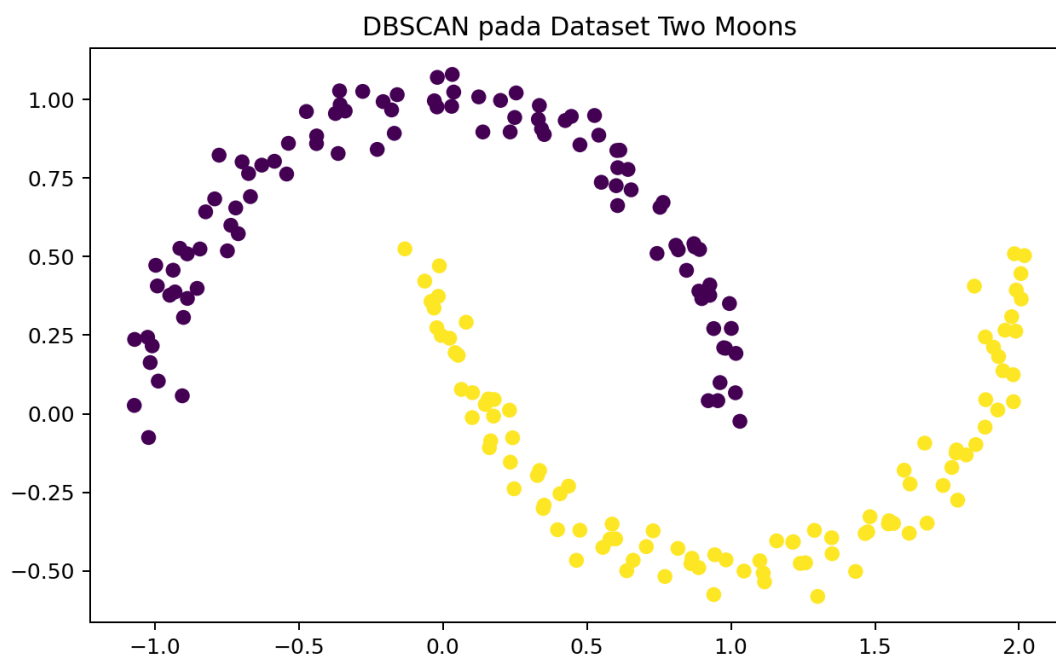
Sumber kode: github.com/ALPHARIET/machine-learning/ch3_11_agglomerative.py

2.23 ch3_12_dbscan.py – DBSCAN

DBSCAN diterapkan pada dataset `make_moons` dengan noise 0,05, menggunakan `eps=0,3` dan `min_samples=5`. Tidak seperti K-Means dan agglomerative clustering, DBSCAN tidak mengharuskan jumlah cluster ditentukan di awal dan dapat menandai data sebagai noise. Pada eksekusi ini, label yang muncul hanya 0 dan 1.

Output	Nilai
Label cluster	[0, 1]
Jumlah anggota cluster 0	100
Jumlah anggota cluster 1	100
Label noise (-1)	Tidak muncul

Kedua cluster terbentuk dengan masing-masing 100 data dan tidak ada titik yang diberi label noise. Dataset `two moons` dipilih karena memiliki struktur nonlinier yang cocok untuk mendemonstrasikan keunggulan pendekatan berbasis kepadatan dibanding pemisahan yang cenderung berbentuk geometris sederhana.



Gambar 8. Hasil DBSCAN pada dataset two moons.

Sumber kode: github.com/ALPHARIET/machine-learning/ch3_12_dbscan.py

2.24 ch3_13_perbandingan_clustering.py – Perbandingan Clustering

File ini membandingkan K-Means, Agglomerative Clustering, dan DBSCAN pada dataset two moons yang telah distandardisasi. Ketiga algoritma menghasilkan label cluster, kemudian label tersebut dibandingkan terhadap `y_moons` menggunakan Adjusted Rand Index (ARI). Metrik ARI mengukur kesesuaian hasil pengelompokan dengan label referensi sambil menyesuaikan kecocokan yang terjadi secara kebetulan.

Algoritma	ARI
K-Means	0.5016
Agglomerative	0.6065
DBSCAN	1.0000

Pada dataset two moons ini, DBSCAN menghasilkan ARI 1,0000, sedangkan K-Means dan Agglomerative menghasilkan skor lebih rendah. Hasil ini konsisten dengan bentuk data yang melengkung, karena DBSCAN bekerja berdasarkan kepadatan dan tidak memaksakan cluster berbentuk pusat-jarak seperti K-Means. Namun hasil tersebut berlaku pada konfigurasi dataset dan

parameter yang ada di file, bukan sebagai klaim bahwa DBSCAN selalu unggul untuk semua data.

Sumber kode: github.com/ALPHARIET/machine-learning/ch3_13_perbandingan_clustering.py

2.25 splitter.py – Kode Pendukung Pembentukan File

splitter.py tidak menjalankan algoritma machine learning. File ini berfungsi sebagai generator berkas latihan. Kode mendefinisikan blok `common_imports`, beberapa setup data reusable untuk dataset Iris, regresi, `make_blobs`, breast cancer, digits, dan two moons, kemudian menyimpan potongan khusus ke dalam dictionary `files_to_create`. Program akhirnya melakukan loop dan menulis setiap kombinasi template ke file Python yang terpisah.

Komponen	Fungsi
<code>common_imports</code>	Impor pustaka yang dipakai bersama
<code>ch2_base / ch2_scaled / ch2_reg</code>	Template data untuk contoh supervised learning dan regression
<code>ch3_blob / ch3_cancer / ch3_digits / ch3_moons</code>	Template dataset untuk bab preprocessing dan clustering
<code>files_to_create</code>	Pemetaan nama file ke setup data + kode spesifik
Loop penulisan	Membuat file pada <code>base_dir</code> (d:\download)

Salah satu implikasi teknis adalah direktori tujuan bersifat spesifik Windows karena `base_dir` ditetapkan ke `d:\download`. Selain itu, `common_imports` memasukkan banyak pustaka yang tidak semuanya digunakan oleh setiap file hasil. Hal tersebut wajar untuk generator pembelajaran, tetapi pada proyek produksi biasanya impor dapat dipersempit per modul agar lebih bersih. Program mencetak "Files created successfully." setelah proses penulisan selesai.

Sumber kode: github.com/ALPHARIET/machine-learning/splitter.py

Catatan Konsistensi dan Kualitas Implementasi

- Konsistensi reproducibility dijaga melalui `random_state` pada sebagian besar contoh, misalnya 0 untuk train-test split, dataset sintetis, Random Forest, MLP, PCA/NMF, t-SNE, dan K-Means.
- Terdapat pengulangan blok impor pada hampir semua file. Ini memudahkan pembelajaran per file, tetapi menghasilkan beberapa impor yang tidak digunakan oleh program tertentu.
- Preprocessing untuk SVM dan MLP dilakukan dengan `fit_transform` pada data latih dan `transform` pada data uji, yang merupakan pola yang tepat untuk mencegah data uji ikut menentukan parameter scaler.
- Perbandingan model pada `ch2_9` belum sepenuhnya seragam karena hanya SVM yang menggunakan data hasil standardisasi. Hal ini perlu dicatat saat menafsirkan perbandingan.
- PCA dan t-SNE dipakai untuk visualisasi, sedangkan NMF digunakan pada data non-negatif asli. Perbedaan prasyarat ini tercermin langsung pada pilihan data di kode.
- Clustering chapter memperlihatkan tiga pendekatan dengan karakteristik berbeda: K-Means dan Agglomerative membutuhkan jumlah cluster, sedangkan DBSCAN memakai parameter kepadatan dan dapat menghasilkan noise.

BAB III PENUTUP

3.1 Kesimpulan

Berdasarkan penelusuran seluruh berkas pada repositori ALPHARIET/machine-learning, dapat disimpulkan bahwa repositori tersebut tersusun sebagai rangkaian contoh pembelajaran machine learning yang bertahap. Bagian awal mempersiapkan dataset dan mendemonstrasikan klasifikasi menggunakan KNN, Logistic Regression, Naive Bayes, Decision Tree, Random Forest, SVM, serta MLP. Bagian regresi memperlihatkan Linear Regression dan Ridge Regression melalui data sintetis.

Bagian berikutnya memperlihatkan peran preprocessing dan reduksi dimensi melalui StandardScaler, MinMaxScaler, RobustScaler, Normalizer, pengaruh scaling terhadap KNN, PCA, NMF, dan t-SNE. Bagian akhir memperkenalkan clustering dengan K-Means, Agglomerative Clustering, DBSCAN, serta evaluasi pembandingan berbasis ARI. Pada eksekusi laporan, contoh KNN dan Logistic Regression pada Iris memperoleh akurasi 1,0000; scaling meningkatkan akurasi KNN pada dataset breast cancer dari sekitar 92,31% menjadi 94,41%; dan DBSCAN mencapai ARI 1,0000 pada dataset two moons dengan konfigurasi yang diberikan di kode.

Secara keseluruhan, repositori berhasil memperlihatkan hubungan antara preprocessing, pemilihan algoritma, parameter, dan metrik evaluasi melalui kode yang relatif ringkas. File splitter.py melengkapi repositori sebagai alat bantu untuk menghasilkan file-file contoh secara terstruktur.

3.2 Saran

- Menambahkan validasi silang dan beberapa random seed agar perbandingan model tidak bergantung pada satu split data.
- Menyamakan preprocessing pada ch2_9 apabila tujuan utamanya adalah membandingkan model secara metodologis.
- Menambahkan dokumentasi dependensi dan instruksi menjalankan seluruh file agar reproduksi di komputer lain lebih mudah.
- Memisahkan impor per file atau menggunakan modul bersama untuk mengurangi pengulangan dan membuat kode lebih maintainable.

DAFTAR PUSTAKA

ALPHARIET. (2026). machine-learning [Repositori GitHub]. GitHub.
<https://github.com/ALPHARIET/machine-learning>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.